Concepts

STRUCTURE

Self-referential data	*List*	DCIC HtDP
	Tree	MIT 6.102
	Nested doll	HtDP
Decomposition	*Structural*	HtDP LC×2
	Generative	HtDP LYAH LC×4
Base case	*Edge condition*	LYAH LC×3
	Pruned leaf	LC-cn LC×1
Leap of faith new	LC×5	

PROCESS

Return & combine	*Value return*	RV LC×8
	Result composition	SICP LC-cn LC×3
Choice / search	*Branching*	LC-cn LC×4
	Pruning	LC-cn LC×2
	Backtracking	LC-cn
Overlapping subproblems	LC-cn LC×2	
Execution state	*Call stack*	LC-cn LC×2
	Accumulator	*Ours*
Cost	*Per level*	CLRS
	Total	CLRS LC×1

Node Content

Concrete data
Values of the input itself.
MIT 6.102 HtDP LC×15
`[1, 4, 3]`

Call with arguments
The call as written, concrete args.
SICP RV LC×4
`fib 3`

State tuple
Only the changing parameters.
LC-cn LC×3
`(i, k)`

Partial expression
Half-evaluated term, as in a trace.
SICP HtDP LC×2
`(* 6 (f 5))`

Symbolic
Parametric size, as in a recurrence.
CLRS LC×1
$f(n/b)$

Visual Encodings

Nested containment
Calls drawn inside their caller.

- *Box-in-box* — HtDP
- *Brackets* — LYAH LC×2 — `f(a(b(c)))`
- *Treemap* — *Ours*

Tree (node-link)
Calls linked to their subcalls.

- *Call tree* — SICP VisuAlgo LC×3
- *State tree* — LC-cn LC×2
- *Choice tree* — LC-cn LC×1
- *Result tree* new — LC×6

Linear chain
List of nodes and links.

- *Box-and-pointer* — HtDP LC×2
- *Pointer rewiring* new — LC×9

Stack
Frames piled up by depth.
LC-cn LC×2

Substitution trace
Expression rewritten until a value.
SICP HtDP LC×1

Indentation staircase
Depth shown by indentation.
DCIC HtDP LC×1

Table
Args, intermediates and results.
HtDP LC×1

Partition new
Input split into boxed parts.
LC×3
left right

Instance mapping new
Size n aligned to size n−1.
LC×3
n n−1

Metaphor / pictorial
Dolls, mirrors, spirals.
LYAH HtDP LC×1

Flow Direction

Static
Structure only, no direction.
HtDP DCIC LC×5

Downward only
Calls drawn, returns omitted.
VisuAlgo LC-cn LC×2

Upward only
Results merging toward the root.
HtDP? LC×1

Round trip
Down to the base case, then back up.
SICP RV LC×7

Rewrite in place
Time runs down the lines.
SICP HtDP

```
f(3)
⇒ 3·f(2)
⇒ 3·2·f(1)
⇒ 6
```

Annotations

Base-case marker
Marks where recursion stops.
LC-cn LC×4

Return value
Value written on the way back.
RV LC-cn LC×6
= 8

Prune / fail + legend
Dead branches, keyed in a legend.
LC-cn LC×3
× fail

Role colour
Colour by role (pivot, done).
LYAH LC×3
pivot done

Cursor / pointer
Indices into the data.
LC-cn LC×6
i j

Choice label
The decision named on the edge.
LC-cn LC×3
take skip

Call-step label new
Each frame or step numbered.
LC×3
call 1
call 2

Duplicate marker
Repeated subproblems circled.
LC-cn LC×1

Traversal path
Evaluation order traced.
SICP LC-cn LC×1

Elision

Full expansion
Small input, every call drawn.
SICP VisuAlgo LC×2

Unpatterned elision
No inference can be made.
LC-cn LC×1

Patterned elision
Rule implied by a few levels.
CLRS

Black-box leaf
Leaves drawn as opaque blocks.
LC-cn

Deferred call
Subcall left unexpanded.

- *Trace term* — HtDP — `f(rest…)`
- *Done box* new — LC×3 — done

Subdiagrams

Multiple representations
Same recursion, two encodings.
HtDP LC×3
`(cons 1 (cons 2 '()))`

Multiple scenarios
Cases of one step side by side.
LC-cn LC×6
a b

Over time
Snapshots of one run.
HtDP LC-cn *Ours* LC×13

Code Correspondence

Diagram is code Trace written in the program's own syntax. SICP HtDP DCIC	**Side by side** Diagram placed next to the code or rule. LYAH SICP LC×5
Arrows into code Arrows tie parts of the diagram to code. HtDP LC×1	**Shared identifiers** Function and parameter names reused in nodes. SICP RV LC×3
Recurrence only Linked to an equation, not to code. CLRS LC×2	**Prose only** Linked through a written explanation. LC-cn LC×2

LYAH Learn You a Haskell
SICP Structure and Interpretation of Computer Programs
HtDP How to Design Programs
DCIC A Data-Centric Introduction to Computing
CLRS Introduction to Algorithms
MIT 6.102 MIT 6.102 course notes
VisuAlgo visualgo.net
RV recursionvisualizer.com
Ours our own sketches
LC-cn LeetCode.cn, hand-picked solutions
LC×n LeetCode, vote ≥ 5: n of 32 solutions
X? example to confirm
new coding found only in the LeetCode set

Figure X: A design space for recursion diagrams, listing the possible codings for each of the eight dimensions.